

Примеры тестов на Python (pytest)

 Перед работой с примерами тестов, ознакомьтесь с [инструкцией по юнит-тестам!](#)

pytest - новый способ запуска тестов на Python, добавленный в [SBIS SDK](#) версии 23.7000, использующий стороннюю библиотеку [pytest](#).

Указать используемый вариант можно в файле test.entry, используя одно из следующих ключевых слов: **py_pytest** или **pytest_on**.

Пример простейшего test.entry для pytest:

```
1 {  
2     "py_pytest": "test_cloud"  
3 }
```

Особенности pytest

1. Для pytest возможно передать дополнительные параметры запуска, указав в файле test.entry ключевое слово **pytest_args**, например:

```
1 {  
2     "py_pytest": "test_cloud",  
3     "pytest_args": "--disable-warnings --durations=1" # отключить  
                вывод предупреждений, отдельно вывести время самого долгого тестового  
                метода.  
4 }
```

2. Для pytest в каталог resources можно добавить конфигурационный файл pytest.ini в котором также передаются настройки для запуска. Пример файла pytest.ini в котором задаются настройки логирования для фреймворка logging:

```
1 [pytest]  
2 log_level = INFO  
3 log_format = %(asctime)s %(levelname)s %(message)s  
4 log_date_format = %Y-%m-%d %H:%M:%S
```

3. Для запуска определенного теста в наборе в аргумент **-t** тестового фреймворка можно передать регулярное выражение вида `"Test_class_name and test_method_name"`, либо относительный путь (относительно директории src) до конкретного файла с тестами `"submodule/test1.py"`.

Наличие файла **init.py** в директории tests/<test_name>/src влияет на выполнение тестов. При его наличии pytest добавляет в sys.path НЕ путь до tests/<test_name>/src, а путь до tests/<test_name>, что может приводить к ошибкам импорта.

Измерение покрытия кода в тестах на Python

В [SBIS SDK](#) версии 23.7000 был добавлен Python-фреймворк [coverage.py](#), который позволяет измерять покрытие кода в питоновских файлах. Он работает с обеими библиотеками pytest и unittest.

Он добавлен в SBIS SDK как самостоятельный python-пакет и лежит в директории tools/python_test_utility. Его можно запустить как самостоятельную программу вместо интерпретатора Python из каталога **{SBIS SDK}/tools/python_test_utility/bin/coverage**.

В тестовый фреймворк добавлена интеграция с coverage.py. Для ее использования в тестовый фреймворк добавлены два параметра:

 Параметры необходимо передавать в строку запуска test_framework.

- **--py_coverage** - при передаче активирует сбор информации о покрытии кода;
- **--py_coverage_format FORMAT** - указать формат отчета (html, xml, json). По умолчанию html.

В файл test.entry введен параметр coverage_args в него можно передать параметры конструктора класса [Coverage](#) в формате json, например:

```
1 "coverage_args" : {
2     "include" : "my_source_dir_name",
3     "concurrency" : "thread"
4 }
```

При его наличии он заменяет параметры, равные по умолчанию:

```
1 {
2     "omit" : [
3         "*/test_framework/*",
4         "*/python_test_utility/*",
5         "*/python_library/*",
6         f"*/{test_dir}/*"
7     ],
8     "concurrency" : "thread"
9 }
```



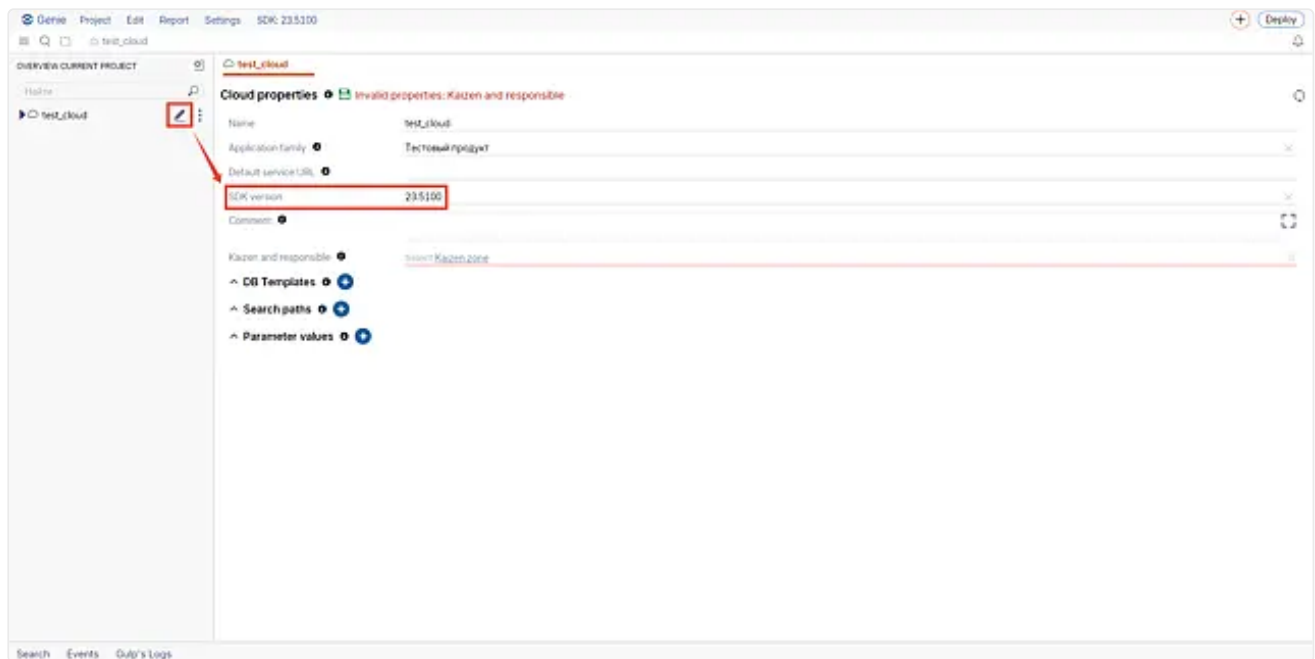
Отчет о покрытии кода по умолчанию генерируется в рабочую директорию теста в папку coverage_{test_name}, либо в папку, указанную в параметре --xunit_output.

Подготовка к запуску тестов

Все примеры тестов выполнены в [SBIS SDK](#) версии 23.5100.

Если вы установили SBIS SDK другой версии, поменяйте ее номер в файле CMakeLists.txt (опционально). Данный файл создается, если у вашего теста есть зависимости от C++ кода не из SDK.

Откройте s3cld-файл проекта в [Genie](#) и в свойствах проверьте номер версии SBIS SDK.



Для запуска тестов в docker-контейнере активируйте виртуальное окружение с помощью команды:

```
1 source /venv_conan/bin/activate (папка в корне)
```

Порядок тестирования и набор тестов аналогичны [примерам тестов на C++](#) — там же можно посмотреть более подробное описание тестов.



При разработке тестов для модулей БЛ скорее всего возникнет проблема "жирного облака", когда тестируемый модуль зависит от множества других напрямую или косвенно. В этом случае рекомендуется часть модулей заменить своими заглушками (пустой модуль с таким же uuid), часть функционала сторонних модулей перекрыть своими реализациями или mock-сервисом (когда нужно перекрыть именно удаленный вызов).

Запуск тестов

Для запуска теста выполните следующие шаги:

1. Откройте командную строку/консоль:

- При работе в Windows откройте [Tensor C++ Developer Command Prompt 2025](#).
- При работе в Linux откройте консоль и с помощью команды export [добавьте переменные окружения](#).

2. Запуск тестирования проекта (команды написаны для SBIS SDK 23.5100). Если запуск выполняется в той же директории, что и файлы тестов, укажите вместо пути к файлам точку.

- Windows 32bit:

```
1 py -3.11-32 "%SBISPlatformSDK_235100%\tools\test_framework" "." ^
2 --jinnee "%SBISPlatformSDK_235100%\tools\jinnee"
```

- Windows 64bit:

```
1 py -3.11-64 "%SBISPlatformSDK_235100%\tools\test_framework" "." ^
2 --jinnee "%SBISPlatformSDK_235100%\tools\jinnee"
```

- Linux:

```
1 /opt/python3.11/bin/python3.11
  $SBISPlatformSDK_235100/tools/test_framework . \
2 --jinnee $SBISPlatformSDK_235100/tools/jinnee
```

- macOS:

```
1 python3.11 $SBISPlatformSDK_235100/tools/test_framework . \
2 --jinnee $SBISPlatformSDK_235100/tools/jinnee
```

Простой тест



[Архив с файлами для запуска теста](#)

Файловая структура каталога теста имеет следующий вид:

```
├─ my_project
│   └─ tests
│       └─ simple_test_suite
│           ├── clouds // каталог для метаданных проекта
│           │   └─ test_cloud // каталог тестового приложения
│           │       ├── test_cloud.s3cld // тестовое облако
│           │       └─ test.s3srv // тестовый сервис
│           ├── src // каталог с исходниками тестов
│           │   └─ simple_test.py // исходный код теста
│           └─ test.entry // файл описания теста
```

В каталоге **clouds** располагается каталог test_cloud с метаданными проекта - файлом облака test_cloud.s3cld и тестовым сервисом test.s3srv. Имя каталога должно совпадать с именем s3cld-файла.

В каталоге **src** расположен файл simple_test.py, в котором размещен исходный код теста.

```
1 import sbis_root as sbis
2 import os
3 from unittest import TestCase
4
5 class TextFrameworkExample(TestCase):
6
7     def test_simple(self):
8         self.assertEqual(1, 1)
```

Файл test.entry будет выглядеть следующим образом:

```
1 {
2     "pytest_on": "test_cloud"
3 }
```

Простой тест с использованием ресурсов



Файлы для запуска теста

Архив с файлами для запуска теста можно скачать [здесь](#).

Файловая структура каталога теста имеет следующий вид:

```
├─ my_project
│   └─ tests
│       └─ res_test_suite
│           ├── clouds // каталог для метаданных проекта
│           │   ├── test_cloud // каталог тестового приложения
│           │   │   ├── test_cloud.s3cld // тестовое облако
│           │   │   └─ test.s3srv // тестовый сервис
│           ├── resources // каталог для дополнительных файлов
│           │   └─ hello_world.txt // текстовый файл
│           ├── src // каталог с исходниками тестов
│           │   └─ resources_test.py // исходный код теста теста
│           └─ test.entry // файл описания теста
```

В каталоге **clouds** располагаются метаданные проекта - файл облака test_cloud.s3cld и тестовый сервис test.s3srv.

В каталог **resources** помещаются [дополнительные файлы](#), на которые после запуска теста будут созданы символические ссылки (в данном примере - файл hello_world.txt с текстом 'Hello, World').

В каталоге **src** расположен файл resources_test.py, в котором размещен исходный код теста:

```
1 import sbis_root as sbis
2 import os
3 from unittest import TestCase
4
5 class TextFrameworkExample(TestCase):
6
7     def test_resources(self):
8         self.assertTrue( os.path.exists("hello_world.txt"))
9
10        with open("hello_world.txt", "r") as f:
11            readFile = f.read()
12            self.assertEqual(readFile, 'Hello, World?')
```

Файл test.entry будет выглядеть следующим образом:

```
1 {
2     "pytest_on": "test_cloud"
3 }
```

В результате тестирования, содержимое тестируемого файла не соответствует строке теста:

```
C:\python\my_project02\build>py -3.11-32 "XSBISPlatformSDK_235100X\tools\test_framework" "..\my_project\tests" ^
Продолжить? --jinnee "XSBISPlatformSDK_235100X\tools\jinnee"
[1] Running test 'res_test_suite' (1/1) ...
Finding zombie processes in 'C:\python\my_project02\build\tests\res_test_suite\test'...
No zombies found
[1] Deploying service C:\python\my_project02\build\tests\res_test_suite from C:\python\my_project02\my_project\tests\res_test_suite\clouds\test_cloud\test_cloud.s3cld
[1] *** ERRORS FOUND
test_resources (resources_test.TextFrameworkExample.test_resources)
Traceback (most recent call last):
  File "C:\python\my_project02\my_project\tests\res_test_suite\src\resources_test.py", line 14, in test_resources
    self.assertEqual(readFile, 'Hello, World?')
AssertionError: 'Hello, World!' != 'Hello, World?'
- Hello, World!
+ Hello, World?
? ^
.....
[1] *** FAIL
[1] Error while running the test [PyTester finished with non-zero code: [1]]
[-] Failed tests:
[-]   res_test_suite
*** 1 failures detected
```

Замените в файле resources_test.py 'Hello, World?' на 'Hello, World!'. Теперь результат теста положительный.

```
C:\python\my_project02\build>py -3.11-32 "XSBISPlatformSDK_235100X\tools\test_framework" "..\my_project\tests" ^
Продолжить? --jinnee "XSBISPlatformSDK_235100X\tools\jinnee"
[1] Running test 'res_test_suite' (1/1) ...
Finding zombie processes in 'C:\python\my_project02\build\tests\res_test_suite\test'...
No zombies found
[1] Deploying service C:\python\my_project02\build\tests\res_test_suite from C:\python\my_project02\my_project\tests\res_test_suite\clouds\test_cloud\test_cloud.s3cld
Finding zombie processes in 'C:\python\my_project02\build\tests\res_test_suite\clouds\test_cloud\test_cloud.s3cld'...
No zombies found
Finding zombie processes in 'C:\python\my_project02\build\tests\res_test_suite\setup_before'...
No zombies found
[1] *** SUCCESS
*** No errors detected
```

Тест с запуском сайта



[Архив с файлами для запуска теста](#)

[Подробное описание теста с запуском сайтов.](#)

Повторите этот тест на Python.

Если у вас есть структура каталога, где лежит файл test_cloud.s3cld:

```
├─ my_project
│   └─ tests
│       └─ site_test_suite
│           ├── clouds // каталог для метаданных проекта
│           │   ├── SomeModule // каталог тестового модуля
│           │   │   ├── SomeModule.orx // справочник объектов
│           │   │   └─ SomeModule.s3mod // тестовый модуль
│           │   └─ test_cloud // каталог тестового приложения
│           │       ├── test_cloud.s3cld // тестовое облако
│           │       └─ test.s3srv // тестовый сервис
│           ├── resources // каталог для дополнительных файлов
│           └─ sbis-rpc-service.ini // файл с параметрами тестового
облака
│   ├── src // каталог с исходниками тестов
│   └─ service_test.py // исходный код теста
└─ test.entry // файл описания теста
```

а файл test.entry имеет следующий вид:

```
1 {
2     "services" : [
```

```

3         {
4             "cloud": "test_cloud"
5         }
6     ],
7
8     "pytest_on": "test_cloud"
9 }

```

то при развороте будет сформирована структура каталогов:

```

├─ build
│   └─ tests
│       └─ site_test_suite
│           └─ run
│               └─ sbis-rpc-service.ini
├─ my_project
│   └─ tests
│       └─ SiteTestSuite
│           ├── clouds
│           │   ├── SomeModule
│           │   │   ├── SomeModule.orx
│           │   │   └─ SomeModule.s3mod
│           │   └─ test_cloud
│           │       ├── test_cloud.s3cld
│           │       └─ test.s3srv
│           ├── resources
│           │   └─ sbis-rpc-service.ini
│           ├── src
│           │   └─ service_test.py
│           └─ test.entry

```

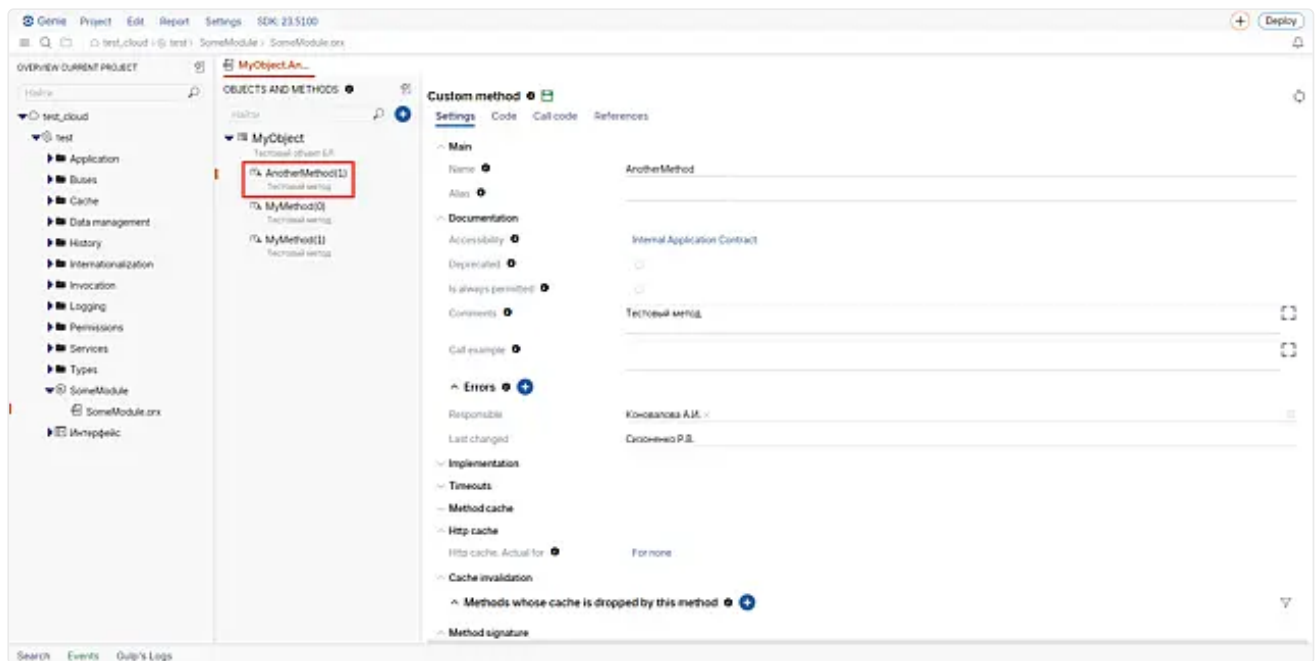
и в файл sbis-rpc-service.ini каталога **run** будут добавлены секции:

```

1  [test]
2  Адрес = http://r02-sizonenko1.region.tensor.ru:10000
3  БазаДанных = postgresql: host='localhost'
4                      port='5432'
5                      dbname='_sbis_test_db_2023-04-02T22-14-13-
6                      062939'
7                      user='postgres'
                          password='<пароль вашей локальной БД>'

```

Откройте проект test_cloud из каталога **clouds** с помощью [Genie](#). В модуле SomeModule в объекте MyObject добавлен метод AnotherMethod:



В каталоге **src** расположены файл `service_test.py`, в котором размещен исходный код теста:

```

1 import sbis_root as sbis
2 import os
3 from unittest import TestCase
4
5 class TextFrameworkExample(TestCase):
6
7     def test_call_service(self):
8         text = sbis.BLObject("MyObject",
9 "test").Invoke("AnotherMethod", 333)
10         self.assertEqual( text, "SuperText" )

```

Тест с использованием Redis

 [Архив с файлами для запуска теста](#)

Файловая структура каталога теста имеет следующий вид:

```

├─ my_project
│   └─ tests
│       └─ redis_test_suite
│           ├── clouds // каталог для метаданных проекта
│           │   ├── test_cloud // каталог тестового приложения
│           │   │   ├── test_cloud.s3cld // тестовое облако
│           │   │   └─ test.s3srv // тестовый сервис
│           ├── src // каталог с исходниками тестов
│           │   └─ redis_test.py // исходный код теста
│           └─ test.entry // файл описания теста

```

В каталоге **clouds** располагаются метаданные проекта - файл облака `test_cloud.s3cld` и тестовый сервис `test.s3srv`.

В каталоге **src** расположен файл `simple_test.py`, в котором размещен исходный код теста

```

1 import sbis_root as sbis
2 import os
3 from unittest import TestCase
4

```

```

5 class TextFrameworkExample(TestCase):
6
7     def test_redis(self):
8         redisClient = sbis.RedisClient ( "Тест.Хранилище", True )
9         key = "123"
10        key_exists = redisClient.IsExists( key );
11        self.assertEqual( key_exists, False)
12
13        redisClient.SetValue(key, "Value")
14        value = redisClient.Get( key );
15        self.assertEqual(value, "Value")

```

Для того чтобы использовать Redis в юнит-тестах, укажите в файле test.entry имя хранилища Redis:

```

1 {
2     "pytest_on": "test_cloud",
3     "redis" : ["Тест.Хранилище"]
4 }

```

Тест с использованием тоск-сервиса

[Подробное описание теста с использованием тоск-сервиса](#)

Тест с использованием БД



[Архив с файлами для запуска теста](#)

[Подробное описание теста с запуском сайтов](#)

Повторите этот тест на Python.

Файловая структура каталога теста имеет следующий вид:

```

├─ my_project
│   └─ tests
│       └─ database_test_suite
│           ├── clouds // каталог для метаданных проекта
│           │   ├── SomeModule // каталог тестового модуля
│           │   │   ├── SomeModule.dicx // словарь данных
│           │   │   ├── SomeModule.dpack // пакет данных
│           │   │   └─ SomeModule.s3mod // файл тестового модуля
│           │   └─ test_cloud // каталог тестового приложения
│           │       ├── test_cloud.s3cld // файл тестового облака
│           │       └─ test.s3srv // тестовый сервис
│           ├── resources // каталог для дополнительных файлов
│           └─ sbis-rpc-service.ini // файл с параметрами тестового
облака
│       ├── src // каталог с исходниками тестов
│       │   └─ service_test.py // исходный код
│       └─ test.entry // файл описания теста

```

Файл тестового облака test_cloud открывается в [Genie](#). В настройках проекта в параметре **DB Templates** создана схема развертывания БД test_db.

В облаке проекта test_cloud создается тестовый модуль SomeModule.s3mod, который содержит:

- [Словарь данных](#) SomeModule с таблицей ТестоваяТаблица.

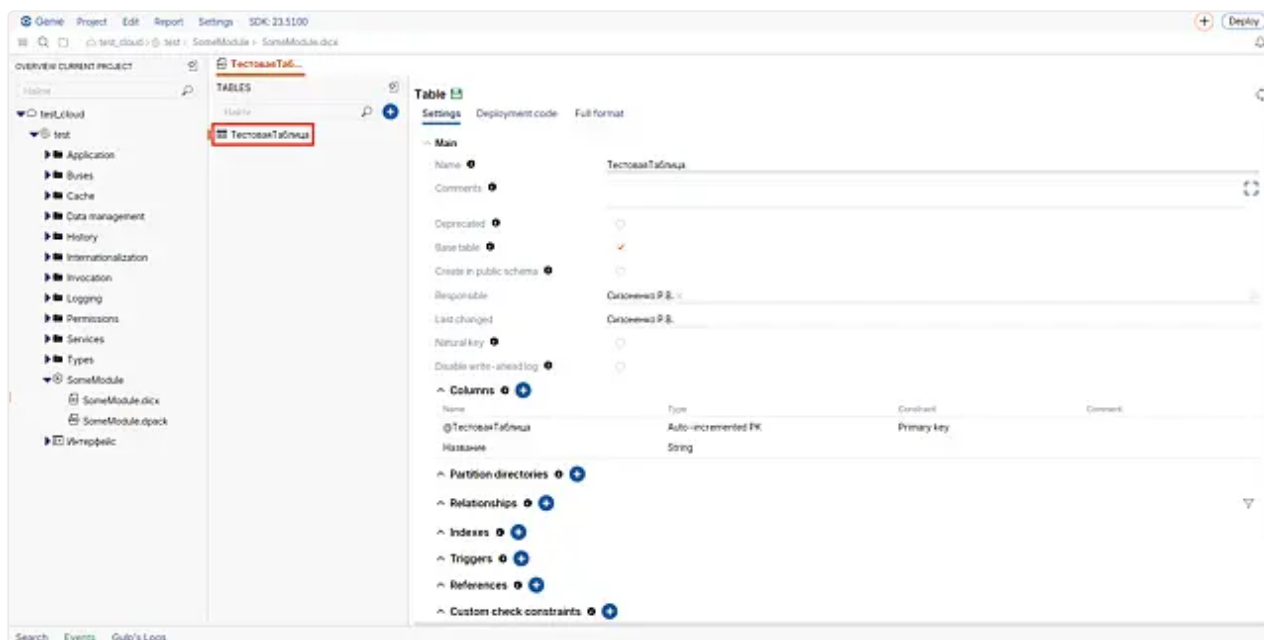
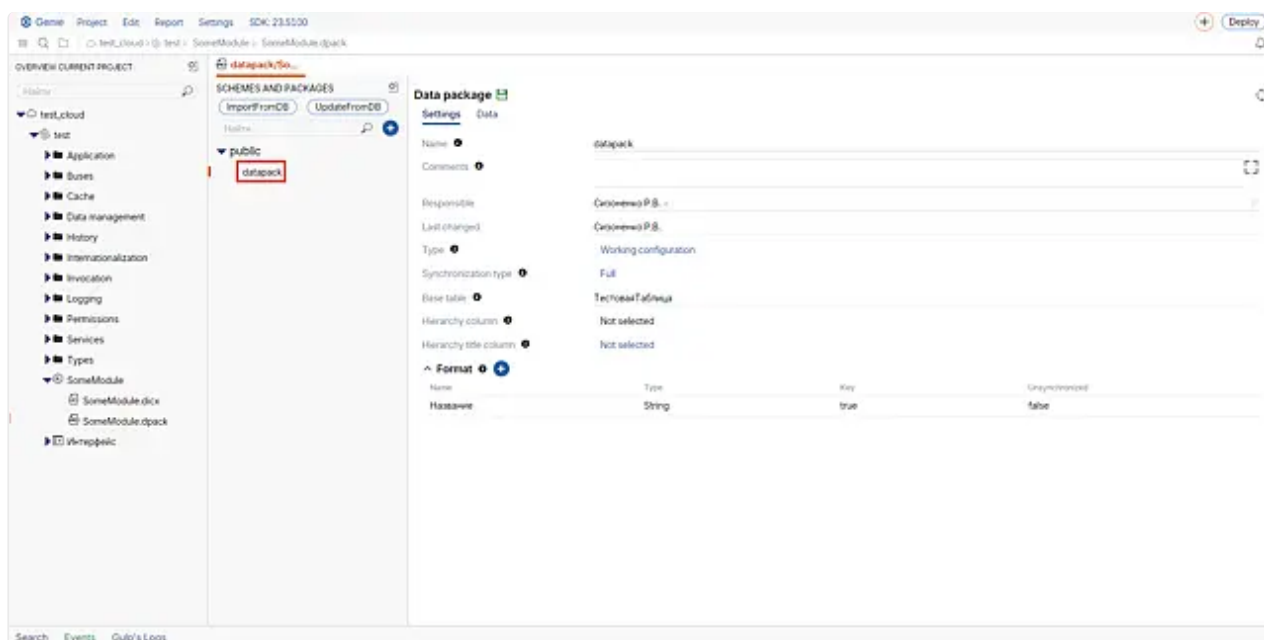
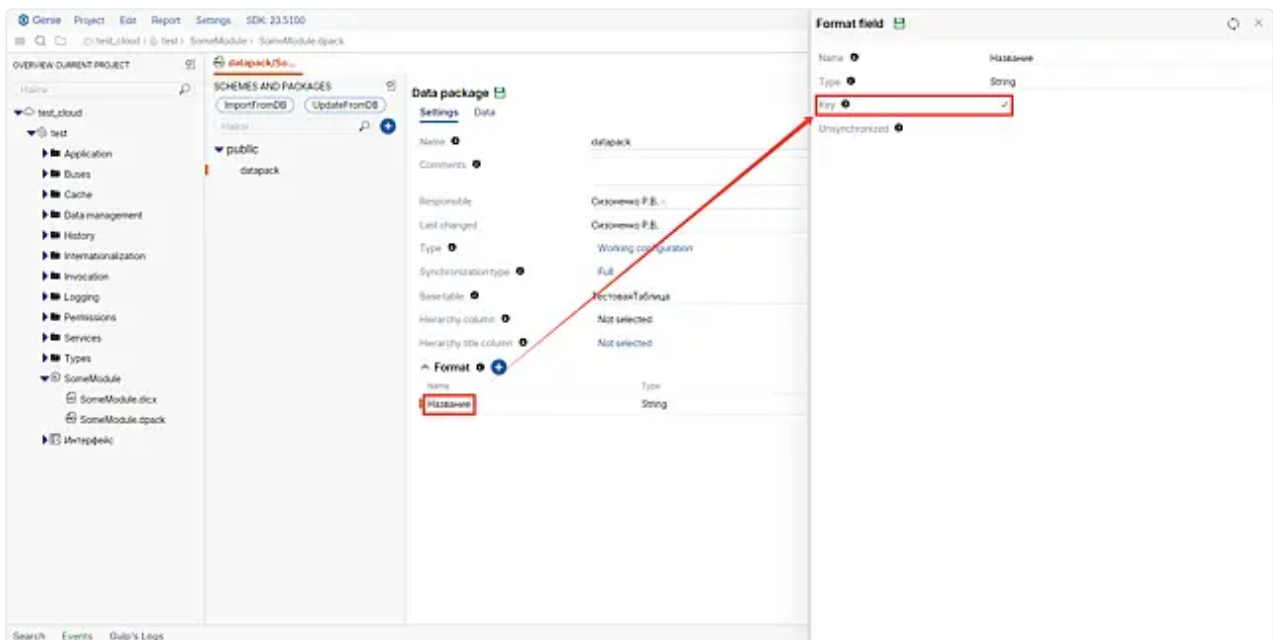


Таблица содержит колонку с именем Название и типом данных String.

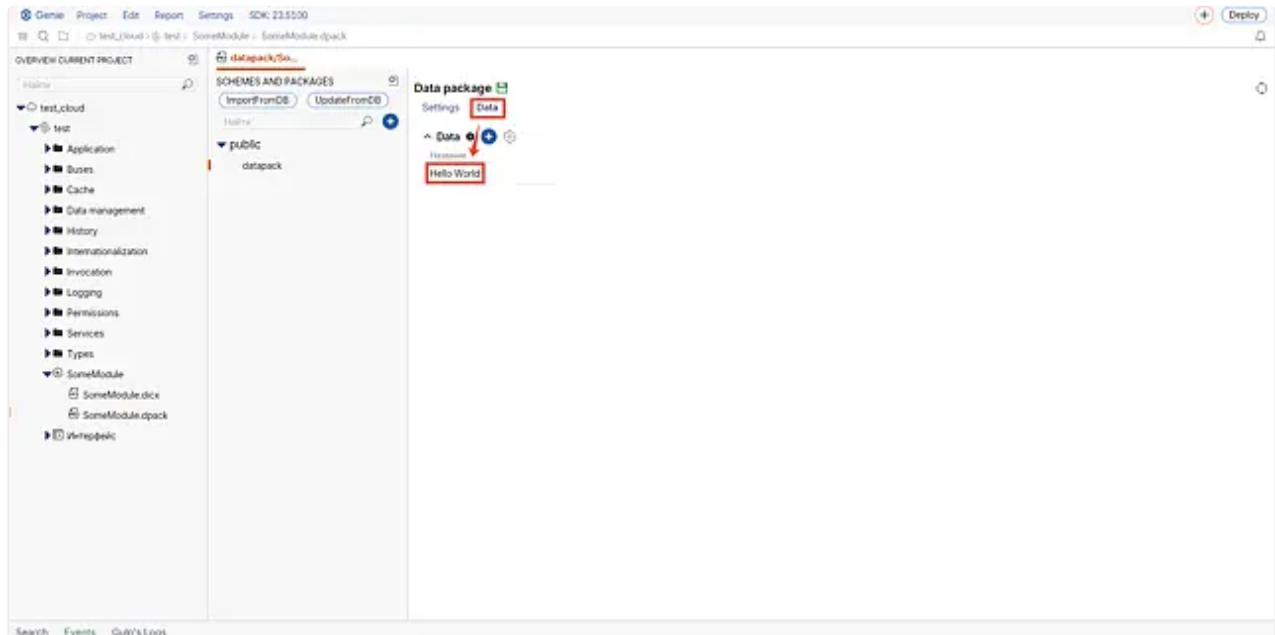
- [Пакет данных](#) SomeModule с пакетом datapack.



Тип синхронизации Full, а в атрибуте **Initialize format from table** указана таблица ТестоваяТаблица. В атрибуте **Format** для колонки Название указан флаг **Key**.



На вкладке пакета данных **Data** добавлено текстовое значение Hello World:



Файл test.entry имеет следующий вид:

```

1 {
2     "pytest_on": "test_cloud"
3 }
```

В каталоге **src** расположены файл service_test.py, в котором размещен исходный код теста:

```

1 import sbis_root as sbis
2 from unittest import TestCase
3
4 class TextFrameworkExample(TestCase):
5
6     def test_args(self):
7         db = sbis.GetConnectedDatabase()
8         res = sbis.SqlQuery( "SELECT \"Название\" FROM
9             \"ТестоваяТаблица\"" )
10         self.assertEqual(res.Size(), 1)
```

Пример содержимого теста на pytest с использованием monkeypatch для подмены endpoint

Рабочий пример теста находится в репозитории [sbis/core](#).

Подробнее про monkeypatch можно прочитать в [официальной документации](#).

```
1  from sbis_root import *
2  import sbis
3  import pytest
4
5
6  #-----
7  # пример подмены удаленного вызова на локальный
8
9  def function_undertest_example1():
10     # функция делает удаленный валидный вызов
11     return sbis.EndPoint("not_available").TestObject.Echo("my_test")
12
13
14  def test_monkeypatching_endpoint(monkeypatch):
15     local_endpoint = sbis.EndPoint("")
16     def mock_endpoint(name, *args, **kwargs):
17         if name == "not_available":
18             return local_endpoint
19         raise Exception("Only mock 'not_available' endpoint")
20     # подменяем endpoint
21     monkeypatch.setattr(sbis, "EndPoint", mock_endpoint)
22
23     # вызываем функцию которую тестируем
24     assert function_undertest_example1() == "my_test"
25
26
27  #-----
28  # подменяем результат вызова метода, метод должен быть доступен
29
30  def function_undertest_example2():
31     # функция делает валидный вызов
32     return TestObject.Echo("test")
33
34
35  def test_monkeypatch_invoke(monkeypatch):
36     def mock_echo(msg, *args, **kwargs):
37         return f"my_test_answer"
38
39     # подмена вызова на локальную функцию
40     monkeypatch.setattr(sbis.TestObject, "Echo", mock_echo)
41
42     # вызываем функцию, которую тестируем
43     assert function_undertest_example2() == "my_test_answer"
44
45
46  #-----
```

```
-----
47 # полная подмена вызова, с созданием ложного метода
48
49 def function_undertest_example3():
50     # функция делает какой-то вызов
51     return
52     sbis.EndPoint("NonExistEndpoint").NonExistTestObject.NonExistTestMetod(
53         10, 10)
54
55 class NonExistTestObject:
56     def NonExistTestMetod(self, a,b):
57         return a*b
58
59 class NonExistEndpoint:
60     NonExistTestObject = NonExistTestObject()
61
62 def test_monkeypatch_all(monkeypatch):
63     def mock_endpoint(name, *args, **kwargs):
64         if name == "NonExistEndpoint":
65             return NonExistEndpoint()
66             raise Exception("Only mock 'NonExistEndpoint' endpoint")
67
68     # подмена endpoint
69     monkeypatch.setattr(sbis, "EndPoint", mock_endpoint)
70
71     assert function_undertest_example3() == 100
```